

第8章: 編集・削除・詳細表示機能の実装

前章で作った「一覧表示（Read）」と「新規登録（Create）」に続いて、この章では残りの機能を全て実装します。この章を終えると、**CRUDの全機能が揃った完動するアプリ**が完成します！

この章で実装する機能

前章までで、書籍の一覧表示と新規登録機能を実装しました。この章では、CRUD（Create=作成, Read=読み取り, Update=更新, Delete=削除）の残りの機能を実装します。

機能	CRUDのどれ？	ユーザーの操作	技術的に行うこと
詳細表示	R（Read）	書籍カードをクリック → 全情報を表示	動的ルーティング（ <code>/books/[id]</code> ）でSupabaseから1件取得
編集	U（Update）	詳細画面の「編集」ボタン → フォームで修正 → 保存	既存データをフォームに表示し、SupabaseのUPDATEを実行
削除	D（Delete）	「削除」ボタン → 確認ダイアログ → 削除	SupabaseのDELETEを実行し、一覧に戻る
検索・フィルタ	（発展）	キーワード入力やステータスで絞り込み	Supabaseの <code>ilike</code> （部分一致検索）クエリを使用
ソート	（発展）	「新しい順」「評価順」などで並べ替え	Supabaseの <code>order</code> クエリを使用

動的ルーティングとは？ URLの一部を変数として扱う仕組みです。例えば `/books/abc123` と `/books/xyz789` は、同じページ定義（`/books/[id]/page.tsx`）で処理されます。`[id]` の部分がそれぞれ `abc123` や `xyz789` に置き換わります。

目次

- 0. 前提知識: 動的ルーティング・useState・useRouter
- 1. 詳細表示機能
- 2. 編集機能
- 3. 削除機能
- 4. 検索・フィルタ機能（発展）
- 5. ソート機能

- 6. 全機能の動作確認
- 7. トラブルシューティング
- 8. 全体のページ遷移図

0. 前提知識: 動的ルーティング・useState・useRouter

この章では、特定の本を「ID指定で開く・編集する・削除する」機能を作ります。コードに入る前に、3つの仕組みだけ押さえます。

0.1 動的ルーティング (Dynamic Routes)

「URLの中に変数を含めたい」場合に使う Next.js の機能です。

```
app/books/[id]/page.tsx ← フォルダ名を [角括弧] で囲む
```

このファイルがあると、

アクセスされたURL	内部で <code>params.id</code> の値
<code>/books/abc123</code>	<code>"abc123"</code>
<code>/books/xyz789</code>	<code>"xyz789"</code>
<code>/books/42</code>	<code>"42"</code>

のように、`[id]` の部分が変数として取り出せます。Next.js 15 以降では `params` は `Promise` で渡されるため、`await` で取り出す必要があります（後ほどコード内で出てきます）。

0.2 useState の超復習

第3章で出てきた React Hook。「コンポーネントが覚えておきたい状態（変化する値）」を持つために使います。

```
"use client"; // ← これがないと useState は使えない (クライアントコンポーネント宣言)
import { useState } from "react";

export default function Counter() {
  // [現在の値, 値を更新する関数] = useState(初期値)
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>カウント: {count}</p>
      <button onClick={() => setCount(count + 1)}>+1</button>
    </div>
  );
}
```

▼ 動作: - 最初に画面に「カウント: 0」と表示される - ボタンを押すたびに count が +1 され、画面が再描画される

0.3 useRouter で画面遷移

ボタンを押した後にプログラムの別ページに遷移したいときは `useRouter` を使います。

```
"use client";
import { useRouter } from "next/navigation";

export default function MyComponent() {
  const router = useRouter();

  const handleClick = () => {
    router.push("/books"); // /books に遷移する
    router.refresh();      // データを最新化 (必要時のみ)
  };

  return <button onClick={handleClick}>一覧へ戻る</button>;
}
```

▼ よく使うメソッド:

メソッド	用途	例
<code>router.push("/books")</code>	新しいURLに遷移	削除後に一覧へ戻る
<code>router.replace("/login")</code>	履歴を残さず遷移	ログイン誘導
<code>router.back()</code>	ブラウザの戻るボタンと同じ	キャンセル
<code>router.refresh()</code>	現在ページを再取得	データ更新後の反映

0.4 Server Component と Client Component の使い分け（おさらい）

種類	デフォルト/宣言	できること	できないこと
Server Component	デフォルト（何も書かない）	DBに直接アクセス、 <code>await</code> で取得	<code>useState</code> / イベントハンドラ
Client Component	先頭に <code>"use client";</code> を書く	<code>useState</code> 、ボタンの <code>onClick</code> など	DB直接アクセス（API経由になる）

この章での使い分け: - 詳細ページ（読み込みのみ） → Server Component - 編集フォーム（入力に反応させる） → Client Component - 削除ボタン（確認ダイアログ + APIコール） → Client Component

1. 詳細表示機能

書籍の詳細情報を表示するページを作成します。ここでは Next.js の動的ルーティング（Dynamic Routes）を活用します。

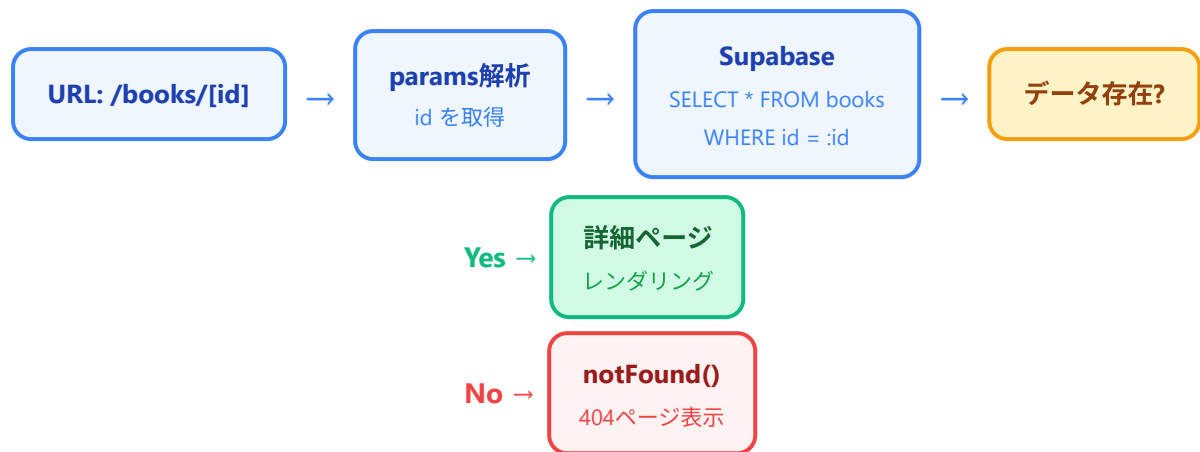
1-1. 動的ルーティングとは

Next.js App Router では、フォルダ名を `[パラメータ名]` のように角括弧で囲むことで、動的なURLセグメントを作成できます。例えば `app/books/[id]/page.tsx` というファイルを作ると、`/books/abc123` や `/books/xyz789` のようなURLにマッチし、`abc123` や `xyz789` の部分を `params.id` として取得できます。

```
app/
  books/
    page.tsx      → /books（一覧ページ）
    new/
      page.tsx    → /books/new（新規登録ページ）
    [id]/
      page.tsx    → /books/:id（詳細ページ）
      edit/
        page.tsx  → /books/:id/edit（編集ページ）
```

1-2. 処理フローの概要

以下のフロー図は、詳細ページにアクセスしたときの処理の流れを示しています。



URLにアクセスすると、Next.js が URL 内の `[id]` 部分を解析して `params` オブジェクトに格納します。そのIDを使って Supabase からデータを取得し、データが存在すれば詳細ページをレンダリング、存在しなければ 404 ページを表示します。

1-3. 書籍詳細ページの作成

まず、`app/books/[id]/` ディレクトリを作成し、その中に `page.tsx` を作成します。

ファイル: `app/books/[id]/page.tsx`

```

import { createClient } from "@lib/supabase/server";
import { notFound } from "next/navigation";
import Link from "next/link";
import DeleteButton from "@components/DeleteButton";

// -----
// 型定義
// -----
// Next.js App Router のページコンポーネントは、
// params プロパティを受け取ります。
// [id] フォルダに配置されているので、params.id で
// URLの動的部分を取得できます。
// -----
type Props = {
  params: Promise<{ id: string }>;
};

// -----
// ステータス表示用のヘルパー関数
// -----
// データベースに保存されている英語のステータス値を
// 日本語の表示ラベルに変換します。
// -----
function getStatusLabel(status: string): string {
  const statusMap: Record<string, string> = {
    unread: "未読",
    reading: "読書中",
    finished: "読了",
  };
  return statusMap[status] || status;
}

// -----
// ステータスに応じたバッジの色を返すヘルパー関数
// -----
// ステータスごとに異なる色のバッジを表示することで、
// ひと目で書籍の読書状態がわかるようにします。
// -----
function getStatusColor(status: string): string {
  switch (status) {
    case "unread":
      return "bg-gray-100 text-gray-800";
    case "reading":
      return "bg-blue-100 text-blue-800";
    case "finished":
      return "bg-green-100 text-green-800";
    default:
      return "bg-gray-100 text-gray-800";
  }
}

```

```

}

// -----
// 評価を星マークで表示するヘルパー関数
// -----
// 数値の評価（1～5）を視覚的な星マーク（★☆）に
// 変換して表示します。
// -----
function renderStars(rating: number | null): string {
  if (rating === null || rating === undefined) return "未評価";
  const filled = "★".repeat(rating);
  const empty = "☆".repeat(5 - rating);
  return filled + empty;
}

// -----
// 書籍詳細ページコンポーネント (Server Component)
// -----
// このコンポーネントはサーバーサイドで実行されます。
// async 関数として定義することで、コンポーネント内で
// 直接 Supabase へのデータ取得を行えます。
// -----
export default async function BookDetailPage({ params }: Props) {
  // -----
  // 1. params から書籍IDを取得
  // -----
  // Next.js 15 以降、params は Promise として渡される
  // ため、await で解決する必要があります。
  // -----
  const { id } = await params;

  // -----
  // 2. Supabase クライアントの作成とデータ取得
  // -----
  // サーバーコンポーネント用の Supabase クライアントを
  // 作成し、指定されたIDの書籍データを取得します。
  // .single() を使うことで、配列ではなく単一の
  // オブジェクトとしてデータを受け取ります。
  // -----
  const supabase = await createClient();

  const { data: book, error } = await supabase
    .from("books")
    .select("*")
    .eq("id", id)
    .single();

  // -----
  // 3. エラーハンドリング
  // -----

```

```

// データが取得できなかった場合 (IDが存在しない、
// ネットワークエラーなど)、Next.js の notFound()
// 関数を呼び出して 404 ページを表示します。
//
// notFound() は例外をスローする関数なので、
// この行以降のコードは実行されません。
// -----
if (error || !book) {
  notFound();
}

// -----
// 4. 日付のフォーマット
// -----
// データベースに保存されている日付文字列を
// 日本語の表示形式に変換します。
// -----
const formatDate = (dateString: string | null): string => {
  if (!dateString) return "未設定";
  const date = new Date(dateString);
  return date.toLocaleDateString("ja-JP", {
    year: "numeric",
    month: "long",
    day: "numeric",
  });
};

// -----
// 5. ページのレンダリング
// -----
// 詳細ページは以下のようなレイアウトで表示されます:
//
// カード形式のレイアウトで、書籍の全情報を
// 見やすく表示します。
// -----
return (
  <div className="max-w-2xl mx-auto py-8 px-4">
    {/* -----
      戻るリンク
      一覧ページに戻るためのリンクです。
      ページ上部に配置して、ユーザーがすぐに
      一覧に戻れるようにします。
      ----- */}

    <Link
      href="/books"
      className="inline-flex items-center text-sm text-blue-600 hover:text-blue-800 mb-6"
    >
      <svg
        className="w-4 h-4 mr-1"
        fill="none"

```



```

        stroke="currentColor"
        viewBox="0 0 24 24"
      >
        <path
          strokeLinecap="round"
          strokeLinejoin="round"
          strokeWidth={2}
          d="M15 19l-7-7 7-7"
        />
      </svg>
      一覧に戻る
    </Link>

    {/* -----
      書籍詳細カード
      白背景のカードに影をつけて、情報を
      グループ化して表示します。
    ----- */}

    <div className="bg-white shadow-lg rounded-lg overflow-hidden">
      {/* -----
        ヘッダー部分
        書籍タイトルとステータスバッジを
        表示します。背景色をグラデーションに
        して視覚的に目立たせます。
      ----- */}

      <div className="bg-gradient-to-r from-blue-600 to-blue-800 px-6 py-8">
        <div className="flex items-start justify-between">
          <h1 className="text-2xl font-bold text-white">{book.title}</h1>
          <span
            className={`inline-flex items-center px-3 py-1 rounded-full text-sm font-medium`}
          >
            {getStatusLabel(book.status)}
          </span>
        </div>
        <div>
          {book.author} && (
            <p className="mt-2 text-blue-100 text-lg">{book.author}</p>
          )
        </div>
      </div>

      {/* -----
        詳細情報部分
        書籍の各項目を定義リスト風のレイアウトで
        表示します。ラベルと値を横並びにして、
        見やすく整理します。
      ----- */}

      <div className="px-6 py-6">
        <dl className="divide-y divide-gray-200">
          {/* 出版社 */}
          <div className="py-4 sm:grid sm:grid-cols-3 sm:gap-4">
            <dt className="text-sm font-medium text-gray-500">出版社</dt>

```

```

        <dd className="mt-1 text-sm text-gray-900 sm:col-span-2 sm:mt-0">
            {book.publisher || "未設定"}
        </dd>
    </div>

    {/* 出版日 */}
    <div className="py-4 sm:grid sm:grid-cols-3 sm:gap-4">
        <dt className="text-sm font-medium text-gray-500">出版日</dt>
        <dd className="mt-1 text-sm text-gray-900 sm:col-span-2 sm:mt-0">
            {formatDate(book.published_date)}
        </dd>
    </div>

    {/* 評価 */}
    <div className="py-4 sm:grid sm:grid-cols-3 sm:gap-4">
        <dt className="text-sm font-medium text-gray-500">評価</dt>
        <dd className="mt-1 text-sm text-gray-900 sm:col-span-2 sm:mt-0">
            <span className="text-yellow-500 text-lg">
                {renderStars(book.rating)}
            </span>
        </dd>
    </div>

    {/* ステータス */}
    <div className="py-4 sm:grid sm:grid-cols-3 sm:gap-4">
        <dt className="text-sm font-medium text-gray-500">ステータス</dt>
        <dd className="mt-1 text-sm text-gray-900 sm:col-span-2 sm:mt-0">
            <span
                className={`inline-flex items-center px-2.5 py-0.5 rounded-full text-xs`}
                >
                {getStatusLabel(book.status)}
            </span>
        </dd>
    </div>

    {/* メモ */}
    <div className="py-4 sm:grid sm:grid-cols-3 sm:gap-4">
        <dt className="text-sm font-medium text-gray-500">メモ</dt>
        <dd className="mt-1 text-sm text-gray-900 sm:col-span-2 sm:mt-0">
            {book.memo ? (
                <div className="bg-gray-50 rounded-md p-4 whitespace-pre-wrap">
                    {book.memo}
                </div>
            ) : (
                <span className="text-gray-400">メモはありません</span>
            )}
        </dd>
    </div>

    {/* 登録日時 */}

```

```

    <div className="py-4 sm:grid sm:grid-cols-3 sm:gap-4">
      <dt className="text-sm font-medium text-gray-500">登録日時</dt>
      <dd className="mt-1 text-sm text-gray-900 sm:col-span-2 sm:mt-0">
        {formatDate(book.created_at)}
      </dd>
    </div>
  </dl>
</div>

  {/* -----
    アクションボタン部分
    編集と削除の操作ボタンを表示します。
    編集はリンク（青色）、削除はボタン
    （赤色）で表示して、操作を視覚的に
    区別します。
    ----- */}

  <div className="bg-gray-50 px-6 py-4 flex items-center justify-end space-x-3">
    <Link
      href={`\books/${book.id}/edit`}
      className="inline-flex items-center px-4 py-2 border border-transparent text-s
    >
      <svg
        className="w-4 h-4 mr-2"
        fill="none"
        stroke="currentColor"
        viewBox="0 0 24 24"
      >
        <path
          strokeLinecap="round"
          strokeLinejoin="round"
          strokeWidth={2}
          d="M11 5H6a2 2 0 0-2 2v11a2 2 0 02 2h11a2 2 0 02-2v-5m-1.414-9.414a2 2
        />
      </svg>
      編集する
    </Link>

    {/* -----
      DeleteButton はクライアントコンポーネント
      です。window.confirm による確認ダイアログ
      やクリックイベントの処理はブラウザ側で
      行う必要があるためです。
      ----- */}

    <DeleteButton bookId={book.id} bookTitle={book.title} />
  </div>
</div>
</div>
);
}

```

このコードのポイント:

- `params` は Next.js 15 以降で `Promise` として渡されるため、`await params` で値を取得します。古いバージョンの Next.js では直接 `params.id` でアクセスでしたが、最新版ではこの非同期パターンが必要です。
- `notFound()` は `next/navigation` から import する関数で、呼び出すと即座に 404 ページが表示されます。内部的には例外をスローするため、`notFound()` 以降のコードは実行されません。
- `.single()` メソッドは、クエリ結果が正確に1件であることを期待します。0件や2件以上の場合はエラーになります。
- サーバーコンポーネントなので `"use client"` ディレクティブは不要です。データ取得はすべてサーバーサイドで行われ、完成した HTML がクライアントに送信されます。

詳細ページは上記のコード内のコメントで示した通り、カード形式のレイアウトで表示されます。完成イメージは以下の通りです:

[← 一覧に戻る](#)



書籍タイトル

著者	山田太郎
出版社	技術評論社
出版日	2024年1月15日
評価	★★★★☆
ステータス	 読書中

メモ

とても良い本です。特に第3章が参考になりました。実務でも活用できる内容が多いです。

 編集する

 削除する

ページ上部にはグラデーション背景のヘッダーがあり、書籍タイトル、著者名、ステータスバッジが表示されます。その下に出版社、出版日、評価（星マーク）、ステータス、メモ、登録日時が定義リスト風に並びます。ページ最下部にはグレー背景のフッターに「編集する」ボタン（青色）と「削除する」ボタン（赤色）が配置されます。

2. 編集機能

書籍情報を編集する機能を実装します。編集機能では、前章で作成した **BookForm** コンポーネントを再利用し、既存データを初期値としてフォームに表示します。

2-1. 編集処理のフロー

Phase 1: データ読み込み

- 1 ユーザー → ブラウザ: `/books/[id]/edit` にアクセス
- 2 編集ページ → **Supabase**: `SELECT * FROM books WHERE id = :id`
- 3 **Supabase** → 編集ページ → ブラウザ: フォーム（既存データ入り）をレンダリング

--- ユーザーがフォームを編集 ---

Phase 2: データ更新

- 4 ユーザー: 「更新」 ボタンをクリック
- 5 ブラウザ → **Supabase**: `UPDATE books SET ... WHERE id = :id`
- 6 更新完了: `router.push("/books/[id]")` → 詳細ページにリダイレクト

このフロー図が示すように、編集機能は2段階の処理で構成されています。まずページアクセス時にサーバーサイドで既存データを取得し、フォームの初期値として設定します。次にユーザーがフォームを編集して「更新」ボタンを押すと、クライアントサイドで Supabase の UPDATE クエリが実行され、完了後に詳細ページへリダイレクトされます。

2-2. BookForm の更新（編集対応）

前章で作成した **BookForm** コンポーネントを、新規登録と編集の両方に対応できるように更新します。主な変更点は以下の通りです。

- **initialData** prop: 編集時に既存データをフォームの初期値として渡す
- **isEdit** prop: 新規登録か編集かを区別する
- ボタンテキスト: 新規登録時は「登録する」、編集時は「更新する」
- 送信先の処理: 新規登録時は INSERT、編集時は UPDATE

ファイル: **components/BookForm.tsx**

```

"use client";

import { useState } from "react";
import { useRouter } from "next/navigation";
import { createClient } from "@/lib/supabase/client";

// -----
// 型定義
// -----
// BookFormData: フォームで扱うデータの型
// initialData に渡す型でもあり、フォームの状態
// 管理にも使用します。
// -----
type BookFormData = {
  title: string;
  author: string;
  publisher: string;
  published_date: string;
  rating: number | null;
  status: string;
  memo: string;
};

// -----
// Props の型定義
// -----
// initialData: 編集時に既存のデータを渡すための
//               プロパティ。新規登録時は undefined。
// isEdit:       編集モードかどうかを示すフラグ。
//               true の場合は UPDATE、false の場合は
//               INSERT を実行します。
// bookId:       編集時に対象の書籍IDを渡すための
//               プロパティ。UPDATE の WHERE 句で使用。
// -----
type Props = {
  initialData?: BookFormData;
  isEdit?: boolean;
  bookId?: string;
};

// -----
// デフォルトの初期値
// -----
// 新規登録時に使用するフォームの初期値です。
// すべてのフィールドを空またはデフォルト値で
// 初期化します。
// -----
const defaultFormData: BookFormData = {
  title: "",

```

```

    author: "",
    publisher: "",
    published_date: "",
    rating: null,
    status: "unread",
    memo: "",
  };

export default function BookForm({
  initialData,
  isEdit = false,
  bookId,
}: Props) {
  const router = useRouter();

  // -----
  // フォームの状態管理
  // -----
  // initialData が渡された場合（編集モード）は
  // 既存データを初期値として使用します。
  // 渡されなかった場合（新規登録モード）は
  // defaultFormData を初期値として使用します。
  // -----
  const [formData, setFormData] = useState<BookFormData>(<
    initialData ?? defaultFormData
  >);
  const [isSubmitting, setIsSubmitting] = useState(false);
  const [error, setError] = useState<string | null>(null);

  // -----
  // フォームフィールドの値変更ハンドラ
  // -----
  // 各入力フィールドの onChange イベントで呼ばれ、
  // フォームの状態を更新します。
  // name 属性をキーとして、対応するフィールドの
  // 値だけを更新します。
  // -----
  const handleChange = (
    e: React.ChangeEvent<
      HTMLInputElement | HTMLSelectElement | HTMLTextAreaElement
    >
  ) => {
    const { name, value } = e.target;
    setFormData((prev) => ({
      ...prev,
      [name]: value,
    }));
  };

  // -----

```

```

// 評価の変更ハンドラ
// -----
// 評価は数値として扱うため、専用のハンドラを
// 用意します。空文字の場合は null を設定し、
// それ以外は数値に変換します。
// -----
const handleRatingChange = (e: React.ChangeEvent<HTMLSelectElement>) => {
  const value = e.target.value;
  setFormData((prev) => ({
    ...prev,
    rating: value === "" ? null : parseInt(value, 10),
  }));
};

// -----
// フォーム送信ハンドラ
// -----
// isEdit フラグに応じて INSERT または UPDATE を
// 実行します。
// -----
const handleSubmit = async (e: React.FormEvent) => {
  e.preventDefault();
  setIsSubmitting(true);
  setError(null);

  // -----
  // バリデーション
  // -----
  // タイトルは必須項目です。空の場合はエラーを
  // 表示して処理を中断します。
  // -----
  if (!formData.title.trim()) {
    setError("タイトルは必須です。");
    setIsSubmitting(false);
    return;
  }

  try {
    const supabase = createClient();

    if (isEdit && bookId) {
      // -----
      // 編集モード: UPDATE
      // -----
      // 既存の書籍データを更新します。
      // .eq("id", bookId) で対象のレコードを指定し、
      // フォームの内容で上書きします。
      // -----
      const { error: updateError } = await supabase
        .from("books")

```



```

.update({
  title: formData.title.trim(),
  author: formData.author.trim() || null,
  publisher: formData.publisher.trim() || null,
  published_date: formData.published_date || null,
  rating: formData.rating,
  status: formData.status,
  memo: formData.memo.trim() || null,
})
.eq("id", bookId);

if (updateError) {
  throw updateError;
}

// -----
// 更新成功: 詳細ページにリダイレクト
// -----
// 更新が完了したら、その書籍の詳細ページに
// 遷移します。ユーザーは更新後の情報を
// すぐに確認できます。
// -----
router.push(`/books/${bookId}`);
router.refresh();
} else {
  // -----
  // 新規登録モード: INSERT
  // -----
  // 新しい書籍データを挿入します。
  // .select() を付けることで、挿入されたデータ
  // (自動生成されたIDを含む) を取得できます。
  // -----
  const { data, error: insertError } = await supabase
    .from("books")
    .insert({
      title: formData.title.trim(),
      author: formData.author.trim() || null,
      publisher: formData.publisher.trim() || null,
      published_date: formData.published_date || null,
      rating: formData.rating,
      status: formData.status,
      memo: formData.memo.trim() || null,
    })
    .select()
    .single();

  if (insertError) {
    throw insertError;
  }
}

```

```

// -----
// 登録成功: 一覧ページにリダイレクト
// -----
router.push("/books");
router.refresh();
}
} catch (err) {
  console.error("保存エラー:", err);
  setError(
    isEdit
      ? "書籍の更新に失敗しました。もう一度お試しください。"
      : "書籍の登録に失敗しました。もう一度お試しください。"
  );
} finally {
  setIsSubmitting(false);
}
};

// -----
// フォームのレンダリング
// -----
return (
  <form onSubmit={handleSubmit} className="space-y-6">
    {/* エラーメッセージ */}
    {error && (
      <div className="bg-red-50 border-l-4 border-red-400 p-4 rounded">
        <div className="flex">
          <div className="flex-shrink-0">
            <svg
              className="h-5 w-5 text-red-400"
              viewBox="0 0 20 20"
              fill="currentColor"
            >
              <path
                fillRule="evenodd"
                d="M10 18a8 8 0 10-16 8 8 8 0 00 16 8z"
                clipRule="evenodd"
              />
            </svg>
          </div>
          <div className="ml-3">
            <p className="text-sm text-red-700">{error}</p>
          </div>
        </div>
      </div>
    )}
    {/* タイトル (必須) */}
    <div>
      <label

```

```

    htmlFor="title"
    className="block text-sm font-medium text-gray-700"
  >
    タイトル <span className="text-red-500">*</span>
  </label>
  <input
    type="text"
    id="title"
    name="title"
    required
    value={formData.title}
    onChange={handleChange}
    className="mt-1 block w-full rounded-md border-gray-300 shadow-sm focus:border-t
    placeholder="書籍のタイトルを入力"
  />
</div>

{/* 著者 */}
<div>
  <label
    htmlFor="author"
    className="block text-sm font-medium text-gray-700"
  >
    著者
  </label>
  <input
    type="text"
    id="author"
    name="author"
    value={formData.author}
    onChange={handleChange}
    className="mt-1 block w-full rounded-md border-gray-300 shadow-sm focus:border-t
    placeholder="著者名を入力"
  />
</div>

{/* 出版社 */}
<div>
  <label
    htmlFor="publisher"
    className="block text-sm font-medium text-gray-700"
  >
    出版社
  </label>
  <input
    type="text"
    id="publisher"
    name="publisher"
    value={formData.publisher}
    onChange={handleChange}

```

```

        className="mt-1 block w-full rounded-md border-gray-300 shadow-sm focus:border-t
        placeholder="出版社名を入力"
    />
</div>

{/* 出版日 */}
<div>
    <label
        htmlFor="published_date"
        className="block text-sm font-medium text-gray-700"
    >
        出版日
    </label>
    <input
        type="date"
        id="published_date"
        name="published_date"
        value={formData.published_date}
        onChange={handleChange}
        className="mt-1 block w-full rounded-md border-gray-300 shadow-sm focus:border-t
    />
</div>

{/* 評価 */}
<div>
    <label
        htmlFor="rating"
        className="block text-sm font-medium text-gray-700"
    >
        評価
    </label>
    <select
        id="rating"
        name="rating"
        value={formData.rating ?? ""}
        onChange={handleRatingChange}
        className="mt-1 block w-full rounded-md border-gray-300 shadow-sm focus:border-t
    >
        <option value="">未評価</option>
        <option value="1">★☆☆☆☆ (1) </option>
        <option value="2">★★☆☆☆ (2) </option>
        <option value="3">★★★☆☆ (3) </option>
        <option value="4">★★★★☆ (4) </option>
        <option value="5">★★★★★ (5) </option>
    </select>
</div>

{/* ステータス */}
<div>
    <label

```

```

      htmlFor="status"
      className="block text-sm font-medium text-gray-700"
    >
      ステータス
    </label>
    <select
      id="status"
      name="status"
      value={formData.status}
      onChange={handleChange}
      className="mt-1 block w-full rounded-md border-gray-300 shadow-sm focus:border-t
    >
      <option value="unread">未読</option>
      <option value="reading">読書中</option>
      <option value="finished">読了</option>
    </select>
  </div>

  {/* ✕モ */}
  <div>
    <label
      htmlFor="memo"
      className="block text-sm font-medium text-gray-700"
    >
      ✕モ
    </label>
    <textarea
      id="memo"
      name="memo"
      rows={4}
      value={formData.memo}
      onChange={handleChange}
      className="mt-1 block w-full rounded-md border-gray-300 shadow-sm focus:border-t
      placeholder="読書✕モや感想を入力"
    />
  </div>

  {/* 送信ボタン */}
  <div className="flex items-center justify-end space-x-3">
    <button
      type="button"
      onClick={() => router.back()}
      className="inline-flex items-center px-4 py-2 border border-gray-300 text-sm for
    >
      キャンセル
    </button>
    <button
      type="submit"
      disabled={isSubmitting}
      className="inline-flex items-center px-4 py-2 border border-transparent text-sm

```

```

    >
    {isSubmitting
      ? isEdit
        ? "更新中..."
        : "登録中..."
      : isEdit
        ? "更新する"
        : "登録する"}
    </button>
  </div>
</form>
);
}

```

BookForm の主な変更点の解説:

1. **initialData** prop: 編集時に既存のデータをフォームの初期値として渡します。**useState** の初期値で **initialData ?? defaultFormData** と記述することで、initialData が渡された場合はそのデータを、渡されなかった場合はデフォルト値を使用します。
2. **isEdit** prop: このフラグが **true** の場合、フォーム送信時に **INSERT** ではなく **UPDATE** を実行します。デフォルト値は **false**（新規登録モード）です。
3. **bookId** prop: 編集時に UPDATE の対象を特定するために必要です。**.eq("id", bookId)** で特定のレコードだけを更新します。
4. ボタンテキストの切り替え: **isEdit** フラグに応じて、ボタンのテキストが「登録する」と「更新する」で切り替わります。送信中の表示も「登録中...」と「更新中...」で異なります。
5. リダイレクト先の違い: 新規登録後は一覧ページ（**/books**）へ、編集後は詳細ページ（**/books/\${bookId}**）へリダイレクトします。

編集フォームにアクセスすると、各入力フィールドに既存のデータが入った状態で表示されます。ユーザーは変更したい項目だけを修正して「更新する」ボタンを押せば、データベースが更新されます。

2-3. 編集ページの作成

app/books/[id]/edit/ ディレクトリを作成し、**page.tsx** を作成します。このページはサーバーコンポーネントとして動作し、既存の書籍データを取得して **BookForm** に渡す役割を持ちます。

ファイル: **app/books/[id]/edit/page.tsx**

```

import { createClient } from "@lib/supabase/server";
import { notFound } from "next/navigation";
import Link from "next/link";
import BookForm from "@components/BookForm";

// -----
// 型定義
// -----
type Props = {
  params: Promise<{ id: string }>;
};

// -----
// 編集ページコンポーネント (Server Component)
// -----
// このコンポーネントは以下の処理を行います:
// 1. URLのパラメータから書籍IDを取得
// 2. Supabase から既存の書籍データを取得
// 3. データが存在しなければ 404 を表示
// 4. BookForm コンポーネントに既存データを渡して
// レンダリング
// -----
export default async function BookEditPage({ params }: Props) {
  // -----
  // 1. params から書籍IDを取得
  // -----
  const { id } = await params;

  // -----
  // 2. Supabase から既存データを取得
  // -----
  const supabase = await createClient();

  const { data: book, error } = await supabase
    .from("books")
    .select("*")
    .eq("id", id)
    .single();

  // -----
  // 3. エラーハンドリング
  // -----
  if (error || !book) {
    notFound();
  }

  // -----
  // 4. BookForm に渡す初期データを整形
  // -----

```

```

// データベースから取得したデータをフォームの型に
// 合わせて整形します。null の値は空文字に変換し、
// フォームの入力フィールドで正しく表示されるように
// します。
// -----
const initialData = {
  title: book.title ?? "",
  author: book.author ?? "",
  publisher: book.publisher ?? "",
  published_date: book.published_date ?? "",
  rating: book.rating ?? null,
  status: book.status ?? "unread",
  memo: book.memo ?? "",
};

// -----
// 5. レンダリング
// -----
return (
  <div className="max-w-2xl mx-auto py-8 px-4">
    {/* 戻るリンク */}
    <Link
      href={`\books/${id}`}
      className="inline-flex items-center text-sm text-blue-600 hover:text-blue-800 mb-6"
    >
      <svg
        className="w-4 h-4 mr-1"
        fill="none"
        stroke="currentColor"
        viewBox="0 0 24 24"
      >
        <path
          strokeLinecap="round"
          strokeLinejoin="round"
          strokeWidth={2}
          d="M15 19l-7-7 7-7"
        />
      </svg>
      詳細に戻る
    </Link>

    {/* ページタイトル */}
    <div className="mb-8">
      <h1 className="text-2xl font-bold text-gray-900">書籍を編集</h1>
      <p className="mt-1 text-sm text-gray-600">
        「{book.title}」の情報を編集します。
      </p>
    </div>

    {/* -----

```



```

BookForm コンポーネント
initialData: 既存の書籍データ
isEdit: true (編集モード)
bookId: 対象の書籍ID

----- */}
<div className="bg-white shadow rounded-lg p-6">
  <BookForm initialData={initialData} isEdit={true} bookId={id} />
</div>
</div>
);
}

```

このコードのポイント:

- 編集ページもサーバーコンポーネントです。データ取得はサーバーサイドで行い、取得したデータを **BookForm** (クライアントコンポーネント) に props として渡します。
- **initialData** の整形では、データベースの **null** 値を空文字に変換しています。これにより、フォームの入力フィールドが React の controlled component として正しく動作します (**null** を value に渡すと uncontrolled component になる) 。
- 戻るリンクは詳細ページ (**/books/\${id}**) を指しています。一覧ではなく、編集元の詳細ページに戻る方がユーザー体験として自然です。

3. 削除機能

書籍を削除する機能を実装します。削除は取り消しができない操作なので、**確認ダイアログ**を表示してユーザーの意思を再確認してから実行します。

3-1. 削除処理のフロー



削除ボタンを押すと、まず `window.confirm()` による確認ダイアログが表示されます。「本当に "書籍タイトル" を削除しますか？この操作は取り消せません。」というメッセージが表示され、ユーザーが「OK」を選択した場合のみ削除処理が実行されます。「キャンセル」を選択した場合は何も起こりません。

3-2. DeleteButton コンポーネントの作成

削除ボタンはクライアントサイドのインタラクション（クリックイベント、確認ダイアログ）を必要とするため、`"use client"` ディレクティブを付けたクライアントコンポーネントとして作成します。

ファイル: `components/DeleteButton.tsx`

```

"use client";

import { useState } from "react";
import { useRouter } from "next/navigation";
import { createClient } from "@/lib/supabase/client";

// -----
// Props の型定義
// -----
// bookId:      削除対象の書籍ID。
//              Supabase の DELETE クエリの WHERE 句で
//              使用します。
// bookTitle:   確認ダイアログに表示する書籍タイトル。
//              ユーザーが削除対象を確認できるように
//              するために使用します。
// -----
type Props = {
  bookId: string;
  bookTitle: string;
};

export default function DeleteButton({ bookId, bookTitle }: Props) {
  const router = useRouter();

  // -----
  // 状態管理
  // -----
  // isDeleting: 削除処理中かどうかを管理します。
  //              true の間はボタンを無効化し、
  //              テキストを「削除中...」に変更して、
  //              二重送信を防ぎます。
  // -----
  const [isDeleting, setIsDeleting] = useState(false);

  // -----
  // 削除処理ハンドラ
  // -----
  const handleDelete = async () => {
    // -----
    // 1. 確認ダイアログの表示
    // -----
    // window.confirm() はブラウザ標準の確認ダイアログ
    // を表示します。ユーザーが「OK」をクリックすると
    // true、「キャンセル」をクリックすると false を
    // 返します。
    //
    // 確認ダイアログのメッセージには書籍タイトルを
    // 含め、削除対象が明確になるようにします。
    // -----
  }

```

```

const confirmed = window.confirm(
  `本当に「${bookTitle}」を削除しますか？\nこの操作は取り消せません。`
);

// -----
// キャンセルされた場合は何もしない
// -----
if (!confirmed) {
  return;
}

// -----
// 2. 削除処理の実行
// -----
setIsDeleting(true);

try {
  const supabase = createClient();

  // -----
  // Supabase DELETE クエリ
  // -----
  // .from("books") で books テーブルを指定し、
  // .delete() で削除操作を行います。
  // .eq("id", bookId) で削除対象を書籍IDで
  // 絞り込みます。
  //
  // ※ .eq() を付けずに .delete() だけを実行すると
  // テーブルの全データが削除されるので、
  // 必ず WHERE 条件を指定してください。
  // -----
  const { error } = await supabase
    .from("books")
    .delete()
    .eq("id", bookId);

  if (error) {
    throw error;
  }

  // -----
  // 3. 削除成功: 一覧ページにリダイレクト
  // -----
  // 削除した書籍の詳細ページはもう存在しないため、
  // 一覧ページに遷移します。
  // router.refresh() でサーバーコンポーネントの
  // データを再取得し、削除された書籍が一覧から
  // 消えていることを確認できるようにします。
  // -----
  router.push("/books");

```

```

    router.refresh();
  } catch (err) {
    console.error("削除エラー:", err);
    // -----
    // エラーが発生した場合はアラートで通知
    // -----
    alert("書籍の削除に失敗しました。もう一度お試しください。");
    setIsDeleting(false);
  }
};

// -----
// ボタンのレンダリング
// -----
// 削除ボタンは赤色で表示し、危険な操作であることを
// 視覚的に示します。
// 削除処理中はボタンを無効化し、テキストを
// 「削除中…」に変更して、処理中であることを
// ユーザーに伝えます。
// -----
return (
  <button
    onClick={handleDelete}
    disabled={isDeleting}
    className="inline-flex items-center px-4 py-2 border border-transparent text-sm font
  >
    {isDeleting ? (
      <>
        {/* -----
           ローディングスピナー
           削除処理中に表示される回転アニメーション
           です。ユーザーに処理中であることを
           視覚的にフィードバックします。
           ----- */}

        <svg
          className="animate-spin -ml-1 mr-2 h-4 w-4 text-white"
          xmlns="http://www.w3.org/2000/svg"
          fill="none"
          viewBox="0 0 24 24"
        >
          <circle
            className="opacity-25"
            cx="12"
            cy="12"
            r="10"
            stroke="currentColor"
            strokeWidth="4"
          />
          <path
            className="opacity-75"

```

```

        fill="currentColor"
        d="M4 12a8 8 0 018-8V0C5.373 0 0 5.373 0 12h4zm2 5.291A7.962 7.962 0 014 12h4"
      />
    </svg>
    削除中...
  </>
) : (
  <>
    { /* ゴミ箱アイコン */ }
    <svg
      className="w-4 h-4 mr-2"
      fill="none"
      stroke="currentColor"
      viewBox="0 0 24 24"
    >
      <path
        strokeLinecap="round"
        strokeLinejoin="round"
        strokeWidth={2}
        d="M19 7L-.867 12.142A2 2 0 0116.138 21H7.862a2 2 0 01-1.995-1.858L5 7m5 4v6"
      />
    </svg>
    削除する
  </>
)}
</button>
);
}

```

このコードのポイント:

- `"use client"` ディレクティブが必須です。`window.confirm()` や `onClick` イベントハンドラはブラウザ側でしか動作しないためです。
- `isDeleting` 状態によるローディング管理で、二重クリックによる二重削除を防いでいます。ボタンの `disabled` 属性と組み合わせることで、処理中は追加のクリックを受け付けません。
- 削除後は `router.push("/books")` で一覧ページに遷移し、`router.refresh()` でサーバーコンポーネントのキャッシュを更新します。これにより、一覧ページで最新のデータが表示されます。
- エラー時は `alert()` でユーザーに通知し、`isDeleting` を `false` に戻してリトライ可能にしています。

4. 検索・フィルタ機能（発展）

一覧ページに検索バーとステータスフィルタを追加し、大量の書籍データの中から目的の本を素早く見つけられるようにします。

4-1. 検索バーコンポーネント

タイトルや著者名で書籍を検索できるコンポーネントを作成します。URL の検索パラメータ（ `searchParams` ）を活用して、検索状態をURLに反映させます。これにより、検索結果のURLを共有したり、ブラウザの戻るボタンで検索前の状態に戻ったりできます。

ファイル: `components/SearchBar.tsx`

```

"use client";

import { useRouter, useSearchParams, usePathname } from "next/navigation";
import { useState, useEffect, useCallback } from "react";

// -----
// 検索バーコンポーネント
// -----
// このコンポーネントは以下の機能を提供します:
// 1. テキスト入力による検索
// 2. ステータスによるフィルタリング
// 3. URLの検索パラメータとの同期
// 4. デバウンス処理 (入力のたびにクエリを
//    発行しないように、一定時間待ってから検索)
// -----
export default function SearchBar() {
  const router = useRouter();
  const searchParams = useSearchParams();
  const pathname = usePathname();

  // -----
  // URL の検索パラメータから初期値を取得
  // -----
  // ページをリロードしたり、検索結果のURLに直接
  // アクセスしたりした場合でも、検索状態が保持
  // されるようにします。
  // -----
  const [searchQuery, setSearchQuery] = useState(
    searchParams.get("q") ?? ""
  );
  const [statusFilter, setStatusFilter] = useState(
    searchParams.get("status") ?? ""
  );

  // -----
  // URL の検索パラメータを更新する関数
  // -----
  // 検索クエリやフィルタが変更されたときに、
  // URL の検索パラメータを更新してページを
  // 再レンダリングします。
  // -----
  const updateSearchParams = useCallback(
    (query: string, status: string) => {
      const params = new URLSearchParams(searchParams.toString());

      // -----
      // 検索クエリの設定
      // -----
      // 値が空の場合はパラメータを削除し、

```



```

// URL をクリーンに保ちます。
// 例: /books?q=React&status=reading
//      検索クエリが空なら /books?status=reading
// -----
if (query.trim()) {
  params.set("q", query.trim());
} else {
  params.delete("q");
}

// ステータスフィルタの設定
if (status) {
  params.set("status", status);
} else {
  params.delete("status");
}

// -----
// URL を更新
// -----
// router.push() を使って URL を更新すると、
// Next.js が自動的にサーバーコンポーネントを
// 再レンダリングし、新しい searchParams で
// データを再取得します。
// -----
const queryString = params.toString();
const newUrl = queryString ? `${pathname}?${queryString}` : pathname;
router.push(newUrl);
},
[router, pathname, searchParams]
);

// -----
// デバウンス処理
// -----
// ユーザーがキーボードを打つたびに検索クエリを
// 発行すると、不必要なリクエストが大量に発生
// します。デバウンスを使って、ユーザーが入力を
// 止めてから 300ms 後に検索を実行します。
// -----
useEffect(() => {
  const timer = setTimeout(() => {
    updateSearchParams(searchQuery, statusFilter);
  }, 300);

  // -----
  // クリーンアップ関数
  // -----
  // 次の入力が行われた場合、前のタイマーを
  // キャンセルします。これにより、最後の入力

```

```

// から 300ms 後にのみ検索が実行されます。
// -----
return () => clearTimeout(timer);
}, [searchQuery, statusFilter, updateSearchParams]);

// -----
// 検索条件クリア
// -----
const handleClear = () => {
  setSearchQuery("");
  setStatusFilter("");
  router.push(pathname);
};

// -----
// レンダリング
// -----
return (
  <div className="bg-white shadow rounded-lg p-4 mb-6">
    <div className="flex flex-col sm:flex-row gap-4">
      {/* 検索入力フィールド */}
      <div className="flex-1">
        <label htmlFor="search" className="sr-only">
          検索
        </label>
        <div className="relative">
          {/* 虫眼鏡アイコン */}
          <div className="absolute inset-y-0 left-0 pl-3 flex items-center pointer-events-none">
            <svg
              className="h-5 w-5 text-gray-400"
              fill="none"
              stroke="currentColor"
              viewBox="0 0 24 24"
            >
              <path
                strokeLinecap="round"
                strokeLinejoin="round"
                strokeWidth={2}
                d="M21 21l-6-6m2-5a7 7 0 11-14 0 7 7 0 0114 0z"
              />
            </svg>
          </div>
          <input
            type="text"
            id="search"
            value={searchQuery}
            onChange={(e) => setSearchQuery(e.target.value)}
            placeholder="タイトルまたは著者名で検索..."
            className="block w-full pl-10 pr-3 py-2 border border-gray-300 rounded-md le
          />

```

```

    </div>
  </div>

  { /* ステータスフィルタ */ }
  <div className="sm:w-48">
    <label htmlFor="status-filter" className="sr-only">
      ステータスフィルタ
    </label>
    <select
      id="status-filter"
      value={statusFilter}
      onChange={(e) => setStatusFilter(e.target.value)}
      className="block w-full py-2 px-3 border border-gray-300 bg-white rounded-md s
    >
      <option value="">すべてのステータス</option>
      <option value="unread">未読</option>
      <option value="reading">読書中</option>
      <option value="finished">読了</option>
    </select>
  </div>

  { /* クリアボタン */ }
  {(searchQuery || statusFilter) && (
    <button
      onClick={handleClear}
      className="inline-flex items-center px-3 py-2 border border-gray-300 text-sm +
    >
      <svg
        className="w-4 h-4 mr-1"
        fill="none"
        stroke="currentColor"
        viewBox="0 0 24 24"
      >
        <path
          strokeLinecap="round"
          strokeLinejoin="round"
          strokeWidth={2}
          d="M6 18L18 6M6 6L12 12"
        />
      </svg>
      クリア
    </button>
  )}
</div>
</div>
);
}

```

4-2. 一覧ページの更新（検索・フィルタ対応）

検索バーコンポーネントを一覧ページに組み込み、Supabase のクエリを検索条件に対応させます。

ファイル: `app/books/page.tsx`（更新版）

```

import { createClient } from "@lib/supabase/server";
import Link from "next/link";
import SearchBar from "@components/SearchBar";
import SortSelect from "@components/SortSelect";

// -----
// 型定義
// -----
// Next.js App Router のページコンポーネントは、
// searchParams プロパティでURLの検索パラメータを
// 受け取ることができます。
// -----
type Props = {
  searchParams: Promise<{
    q?: string;
    status?: string;
    sort?: string;
    order?: string;
  }>;
};

// -----
// ステータス表示用のヘルパー関数
// -----
function getStatusLabel(status: string): string {
  const statusMap: Record<string, string> = {
    unread: "未読",
    reading: "読書中",
    finished: "読了",
  };
  return statusMap[status] || status;
}

function getStatusColor(status: string): string {
  switch (status) {
    case "unread":
      return "bg-gray-100 text-gray-800";
    case "reading":
      return "bg-blue-100 text-blue-800";
    case "finished":
      return "bg-green-100 text-green-800";
    default:
      return "bg-gray-100 text-gray-800";
  }
}

// -----
// 評価を星マークで表示する関数
// -----

```

```

function renderStars(rating: number | null): string {
  if (rating === null || rating === undefined) return "-";
  return "★".repeat(rating) + "☆".repeat(5 - rating);
}

// -----
// 書籍一覧ページ (Server Component)
// -----

export default async function BooksPage({ searchParams }: Props) {
  const { q, status, sort, order } = await searchParams;
  const supabase = await createClient();

  // -----
  // Supabase クエリの構築
  // -----
  // 基本のクエリを作成し、検索条件やフィルタ条件が
  // ある場合は動的にクエリを組み立てます。
  // -----
  let query = supabase.from("books").select("*");

  // -----
  // テキスト検索
  // -----
  // q パラメータが指定されている場合、タイトルまたは
  // 著者名で部分一致検索を行います。
  //
  // Supabase の ilike は大文字小文字を区別しない
  // LIKE 検索です。% はワイルドカードで、
  // %検索語% は「検索語を含む」という意味になります。
  //
  // .or() を使って、タイトルまたは著者名のいずれかに
  // マッチする条件を設定します。
  // -----
  if (q) {
    query = query.or(`title.ilike.>${q}%`, `author.ilike.>${q}%`);
  }

  // -----
  // ステータスフィルタ
  // -----
  // status パラメータが指定されている場合、
  // そのステータスの書籍のみを取得します。
  // -----
  if (status) {
    query = query.eq("status", status);
  }

  // -----
  // ソート
  // -----

```

```

// sort パラメータでソート対象のカラムを、
// order パラメータでソート順（昇順/降順）を
// 指定します。デフォルトは作成日の降順です。
// -----
const sortColumn = sort || "created_at";
const ascending = order === "asc";
query = query.order(sortColumn, { ascending });

// -----
// クエリ実行
// -----
const { data: books, error } = await query;

if (error) {
  console.error("書籍の取得に失敗:", error);
}

// -----
// レンダリング
// -----
return (
  <div className="max-w-4xl mx-auto py-8 px-4">
    { /* ページヘッダー */ }
    <div className="flex items-center justify-between mb-6">
      <h1 className="text-2xl font-bold text-gray-900">書籍一覧</h1>
      <Link
        href="/books/new"
        className="inline-flex items-center px-4 py-2 border border-transparent text-sm
      >
        <svg
          className="w-4 h-4 mr-2"
          fill="none"
          stroke="currentColor"
          viewBox="0 0 24 24"
        >
          <path
            strokeLinecap="round"
            strokeLinejoin="round"
            strokeWidth={2}
            d="M12 4v16m8-8H4"
          />
        </svg>
        新規登録
      </Link>
    </div>

    { /* 検索バーとソート */ }
    <SearchBar />
    <SortSelect />
  </div>
);

```

```

{ /* 検索結果の件数表示 */ }
<p className="text-sm text-gray-500 mb-4">
  {books ? `${books.length}件の書籍が見つかりました` : "読み込み中..."}
  {q && (
    <span className="ml-2">
      (検索: 「{q}」)
    </span>
  )}
  {status && (
    <span className="ml-2">
      (フィルタ: {getStatusLabel(status)})
    </span>
  )}
</p>

{ /* 書籍一覧 */ }
{!books || books.length === 0 ? (
  <div className="text-center py-12 bg-white rounded-lg shadow">
    <svg
      className="mx-auto h-12 w-12 text-gray-400"
      fill="none"
      stroke="currentColor"
      viewBox="0 0 24 24"
    >
      <path
        strokeLinecap="round"
        strokeLinejoin="round"
        strokeWidth={2}
        d="M12 6.253v13m0-13C10.832 5.477 9.246 5 7.5 5S4.168 5.477 3 6.253v13C4.168
      />
    </svg>
    <h3 className="mt-2 text-sm font-medium text-gray-900">
      書籍が見つかりません
    </h3>
    <p className="mt-1 text-sm text-gray-500">
      {q || status
        ? "検索条件を変更してお試しください。"
        : "新しい書籍を登録してみましよう。"}
    </p>
    {!q && !status && (
      <div className="mt-6">
        <Link
          href="/books/new"
          className="inline-flex items-center px-4 py-2 border border-transparent st
        >
          最初の書籍を登録する
        </Link>
      </div>
    )}
  </div>

```



```

) : (
  <div className="bg-white shadow overflow-hidden sm:rounded-lg">
    <ul className="divide-y divide-gray-200">
      {books.map((book) => (
        <li key={book.id}>
          <Link
            href={`\books/${book.id}`}
            className="block hover:bg-gray-50 transition-colors duration-150"
          >
            <div className="px-4 py-4 sm:px-6">
              <div className="flex items-center justify-between">
                <div className="flex-1 min-w-0">
                  <p className="text-sm font-medium text-blue-600 truncate">
                    {book.title}
                  </p>
                  <p className="mt-1 text-sm text-gray-500">
                    {book.author || "著者不明"}
                    {book.publisher && ` / ${book.publisher}`}
                  </p>
                </div>
                <div className="ml-4 flex flex-col items-end space-y-1">
                  <span
                    className={`inline-flex items-center px-2.5 py-0.5 rounded-full`}
                  >
                    {getStatusLabel(book.status)}
                  </span>
                  <span>
                    {book.rating && (
                      <span className="text-yellow-500 text-xs">
                        {renderStars(book.rating)}
                      </span>
                    )}
                  </span>
                </div>
              </div>
            </div>
          </li>
        </li>
      )})}
    </ul>
  </div>
)}
</div>
);
}

```

検索・フィルタ機能のポイント:

- Supabase の `ilike` メソッドは、PostgreSQL の `ILIKE` 演算子に対応しており、大文字小文字を区別しない部分一致検索を行います。`%` はワイルドカードで、`%React%` は「React」を含むすべての文字列にマッチします。

- `.or()` メソッドを使うことで、「タイトルに含まれる または 著者名に含まれる」という OR 条件を設定できます。
 - URLの検索パラメータ（ `searchParams` ）を使うことで、検索状態がURLに反映されます。ブラウザの戻る/進むボタンで検索状態を行き来したり、検索結果のURLを他の人と共有したりできます。
-

5. ソート機能

書籍一覧をさまざまな基準で並べ替えられるソートコンポーネントを作成します。

ファイル: `components/SortSelect.tsx`

```

"use client";

import { useRouter, useSearchParams, usePathname } from "next/navigation";

// -----
// ソート選択コンポーネント
// -----
// 書籍一覧の並び順を変更するためのセレクトボックスです。
// 以下のソート基準に対応しています:
// - 登録日 (新しい順/古い順)
// - タイトル (昇順/降順)
// - 評価 (高い順/低い順)
// -----
export default function SortSelect() {
  const router = useRouter();
  const searchParams = useSearchParams();
  const pathname = usePathname();

  // -----
  // 現在のソート状態をURLから取得
  // -----
  const currentSort = searchParams.get("sort") || "created_at";
  const currentOrder = searchParams.get("order") || "desc";
  const currentValue = `${currentSort}-${currentOrder}`;

  // -----
  // ソート変更ハンドラ
  // -----
  // セレクトボックスの値が変更されたときに呼ばれます。
  // 値は "カラム名-昇順/降順" の形式 (例: "title-asc")
  // になっており、これを分解して URL パラメータに
  // 設定します。
  // -----
  const handleSortChange = (e: React.ChangeEvent<HTMLSelectElement>) => {
    const value = e.target.value;
    const [sort, order] = value.split("-");

    const params = new URLSearchParams(searchParams.toString());
    params.set("sort", sort);
    params.set("order", order);

    router.push(`${pathname}?${params.toString()}`);
  };

  // -----
  // レンダリング
  // -----
  return (
    <div className="flex items-center justify-end mb-4">

```

```

<label
  htmlFor="sort"
  className="text-sm text-gray-500 mr-2"
>
  並び順:
</label>
<select
  id="sort"
  value={currentValue}
  onChange={handleSortChange}
  className="block w-48 py-1.5 px-3 border border-gray-300 bg-white rounded-md shadow"
>
  <option value="created_at-desc">登録日（新しい順） </option>
  <option value="created_at-asc">登録日（古い順） </option>
  <option value="title-asc">タイトル（A→Z） </option>
  <option value="title-desc">タイトル（Z→A） </option>
  <option value="rating-desc">評価（高い順） </option>
  <option value="rating-asc">評価（低い順） </option>
</select>
</div>
);
}

```

ソート機能のポイント:

- Supabase の `.order()` メソッドは、PostgreSQL の `ORDER BY` 句に対応しています。第1引数にカラム名、第2引数のオブジェクトで `ascending: true` を指定すると昇順、`ascending: false`（または省略）で降順になります。
- ソート条件もURLの検索パラメータに反映されるため、検索・フィルタと同様にブラウザの履歴管理やURL共有が可能です。
- 検索・フィルタとソートは独立して動作します。例えば「未読」フィルタをかけた状態で「評価（高い順）」にソートすると、「未読かつ評価の高い順」で表示されます。

6. 全機能の動作確認

ここまでで実装した CRUD 全操作と検索・フィルタ・ソート機能の動作を確認します。以下の手順に沿って、各機能を順番にテストしてください。

6-1. 新規登録のテスト

- ブラウザで `http://localhost:3000/books` にアクセスします
- 右上の「新規登録」ボタンをクリックします
- `http://localhost:3000/books/new` に遷移したことを確認します

4. 以下の情報を入力します: - タイトル: **React実践ガイド** - 著者: **山田太郎** - 出版社: **技術評論社** - 出版日: **2024-01-15** - 評価: **★★★★☆ (4)** - ステータス: **読書中** - メモ: **コンポーネント設計の章が特に参考になった**
5. 「登録する」ボタンをクリックします
6. **期待される結果:** 一覧ページにリダイレクトされ、登録した書籍が一覧に表示されていること
7. 同様に、もう2冊登録します: - タイトル: **TypeScript入門**, 著者: **鈴木花子**, ステータス: **未読**, 評価: **3** - タイトル: **Next.js実践**, 著者: **田中一郎**, ステータス: **読了**, 評価: **5**

6-2. 詳細表示のテスト

- 一覧ページで「React実践ガイド」をクリックします
- 詳細ページに遷移したことを確認します
- 期待される結果:** - URL が **/books/{書籍ID}** の形式であること - タイトル「React実践ガイド」がヘッダーに表示されていること - 著者「山田太郎」がヘッダーの下に表示されていること - ステータス「読書中」のバッジが青色で表示されていること - 出版社「技術評論社」が表示されていること - 出版日「2024年1月15日」が日本語形式で表示されていること - 評価「★★★★☆」が黄色い星で表示されていること - メモの内容が灰色の背景内に表示されていること - 「編集する」ボタン（青色）と「削除する」ボタン（赤色）が表示されていること
- 「一覧に戻る」リンクをクリックして、一覧ページに戻れることを確認します

6-3. 編集のテスト

- 詳細ページに戻り、「編集する」ボタンをクリックします
- 編集ページに遷移したことを確認します
- 期待される結果:** - URL が **/books/{書籍ID}/edit** の形式であること - フォームの各フィールドに既存のデータが入った状態で表示されていること
 - タイトル欄に「React実践ガイド」
 - 著者欄に「山田太郎」
 - 出版社欄に「技術評論社」
 - 出版日が「2024-01-15」
 - 評価が「★★★★☆ (4)」
 - ステータスが「読書中」
 - メモ欄に「コンポーネント設計の章が特に参考になった」
 - ボタンのテキストが「更新する」であること
- 以下の変更を行います: - 評価を「★★★★★ (5)」に変更 - ステータスを「読了」に変更 - メモに「最後まで読了。全体的に素晴らしい内容だった」を追加
- 「更新する」ボタンをクリックします

6. **期待される結果:** 詳細ページにリダイレクトされ、更新された情報が反映されていること - 評価が「★★★★★」に変わっていること - ステータスが「読了」（緑色のバッジ）に変わっていること - メモが更新されていること

6-4. 検索・フィルタのテスト

1. 一覧ページに戻ります
2. 検索バーに「React」と入力します
3. **期待される結果:** - 「React実践ガイド」のみが表示されること - URL に `?q=React` が含まれていること - 「1件の書籍が見つかりました（検索:「React」）」と表示されること
4. 検索バーをクリアし、ステータスフィルタで「未読」を選択します
5. **期待される結果:** - ステータスが「未読」の書籍のみが表示されること - URL に `?status=unread` が含まれていること
6. 「クリア」ボタンをクリックして検索条件をリセットします
7. **期待される結果:** 全書籍が表示されること

6-5. ソートのテスト

1. ソートのセレクトボックスで「評価（高い順）」を選択します
2. **期待される結果:** 評価の高い書籍から順に表示されること
3. 「タイトル（A→Z）」を選択します
4. **期待される結果:** タイトルのアルファベット/五十音順に表示されること

6-6. 削除のテスト

1. 一覧ページから「TypeScript入門」をクリックして詳細ページに移動します
2. 「削除する」ボタンをクリックします
3. **期待される結果:** 確認ダイアログが表示され、「本当に「TypeScript入門」を削除しますか？この操作は取り消せません。」というメッセージが表示されること
4. まず「キャンセル」をクリックします
5. **期待される結果:** 何も変化せず、詳細ページが表示されたままであること
6. 再度「削除する」ボタンをクリックし、今度は「OK」をクリックします
7. **期待される結果:** - 一覧ページにリダイレクトされること - 「TypeScript入門」が一覧から消えていること - 残りの書籍が正しく表示されていること

7. トラブルシューティング

開発中によく遭遇するエラーとその対処法をまとめます。

7-1. params の取得でエラーが発生する

エラーメッセージ:

```
Error: Cannot read properties of undefined (reading 'id')
```

または

```
TypeError: params.then is not a function
```

原因と対処法:

Next.js のバージョンによって `params` の扱いが異なります。

- Next.js 15 以降: `params` は `Promise` として渡されるため、`await params` で解決する必要があります。
- Next.js 14 以前: `params` は通常のオブジェクトとして渡されるため、直接 `params.id` でアクセスします。

```
// Next.js 15 以降
type Props = {
  params: Promise<{ id: string }>;
};

export default async function Page({ params }: Props) {
  const { id } = await params; // await が必要
  // ...
}

// Next.js 14 以前
type Props = {
  params: { id: string };
};

export default async function Page({ params }: Props) {
  const { id } = params; // await 不要
  // ...
}
```

自分のプロジェクトの Next.js バージョンは `package.json` で確認できます。

```
cat package.json | grep next
```

7-2. UPDATE / DELETE が反映されない

症状: UPDATE や DELETE を実行してもエラーは出ないが、データが変更されていない。

原因1: RLS (Row Level Security) ポリシーが設定されていない、または不適切

Supabase では RLS がデフォルトで有効になっています。テーブルに適切なポリシーが設定されていない場合、操作が静かに無視されることがあります。

対処法: Supabase のダッシュボードで RLS ポリシーを確認します。

```
-- Supabase SQL Editor で実行
-- 現在のポリシーを確認
SELECT * FROM pg_policies WHERE tablename = 'books';
```

開発中は、以下のような全許可ポリシーを設定できます（本番環境では適切な認証ベースのポリシーに変更してください）。

```
-- すべての操作を許可するポリシー（開発用）
CREATE POLICY "Allow all operations" ON books
  FOR ALL
  USING (true)
  WITH CHECK (true);
```

原因2: `.eq()` の条件が合っていない

UPDATE や DELETE で `.eq("id", bookId)` の `bookId` が正しくない可能性があります。

対処法: コンソールログで値を確認します。

```
console.log("Updating book with ID:", bookId);
const { data, error } = await supabase
  .from("books")
  .update({ title: "..."} )
  .eq("id", bookId)
  .select();
console.log("Update result:", { data, error });
```

原因3: `router.refresh()` を呼んでいない

Next.js のサーバーコンポーネントはデータをキャッシュするため、データベースを更新しただけでは画面に反映されません。

対処法: `router.push()` の後に `router.refresh()` を呼びます。

```
router.push("/books");
router.refresh(); // サーバーコンポーネントのキャッシュを更新
```

7-3. リダイレクトが動作しない

症状: `router.push()` を呼んでもページが遷移しない。

原因1: サーバーコンポーネント内で `useRouter` を使っている

`useRouter` はクライアントコンポーネント専用のフックです。サーバーコンポーネント内では使用できません。

対処法: サーバーコンポーネントでリダイレクトしたい場合は `redirect()` 関数を使います。

```
// サーバーコンポーネント内でのリダイレクト
import { redirect } from "next/navigation";

export default async function Page() {
  // 何らかの条件でリダイレクト
  redirect("/books");
}
```

原因2: `"use client"` ディレクティブの付け忘れ

`useRouter` や `useState` などの React フックを使用するコンポーネントには `"use client"` ディレクティブが必要です。

```
"use client"; // ファイルの最初に追加

import { useRouter } from "next/navigation";
// ...
```

原因3: try-catch の catch ブロック内で処理が止まっている

エラーが発生して catch ブロックに入ると、リダイレクト処理が実行されません。コンソールでエラーを確認してください。

```
try {
  // ... Supabase 操作
  router.push("/books"); // エラーが発生するとここまで到達しない
} catch (err) {
  console.error("Error:", err); // コンソールでエラーを確認
}
```

7-4. RLS 関連のエラー

エラーメッセージ:

```
{
  code: "42501",
  message: "new row violates row-level security policy for table \"books\""
}
```

原因: RLS ポリシーが書き込み操作を許可していません。

対処法:

1. Supabase ダッシュボードの「Authentication」>「Policies」で、`books` テーブルのポリシーを確認します。

2. 開発段階では、以下の SQL で全操作を許可できます。

```
-- 既存のポリシーを削除（必要に応じて）
DROP POLICY IF EXISTS "Allow all operations" ON books;

-- 全操作許可ポリシーを作成
CREATE POLICY "Allow all operations" ON books
  FOR ALL
  USING (true)
  WITH CHECK (true);
```

3. 本番環境では、認証されたユーザーのみが操作できるようにポリシーを設定します。

```
-- 認証済みユーザーのみ操作を許可
CREATE POLICY "Authenticated users can manage books" ON books
  FOR ALL
  TO authenticated
  USING (auth.uid() = user_id)
  WITH CHECK (auth.uid() = user_id);
```

注意: 上記の本番用ポリシーは、`books` テーブルに `user_id` カラムがあり、ユーザーごとにデータを分離する場合の例です。このチュートリアル の 現段階では `user_id` カラムは未実装なので、開発用の全許可ポリシーを使用してください。

7-5. フォームの初期値が反映されない

症状: 編集ページでフォームを開いても、フィールドが空になっている。

原因: `useState` の初期値が正しく設定されていない可能性があります。

対処法:

1. 編集ページで `initialData` が正しく渡されているか確認します。

```
// app/books/[id]/edit/page.tsx
console.log("Book data from DB:", book);
console.log("Initial data for form:", initialData);
```

2. `BookForm` コンポーネントで `initialData` を受け取れているか確認します。

```
// components/BookForm.tsx
console.log("Received initialData:", initialData);
console.log("Form state:", formData);
```

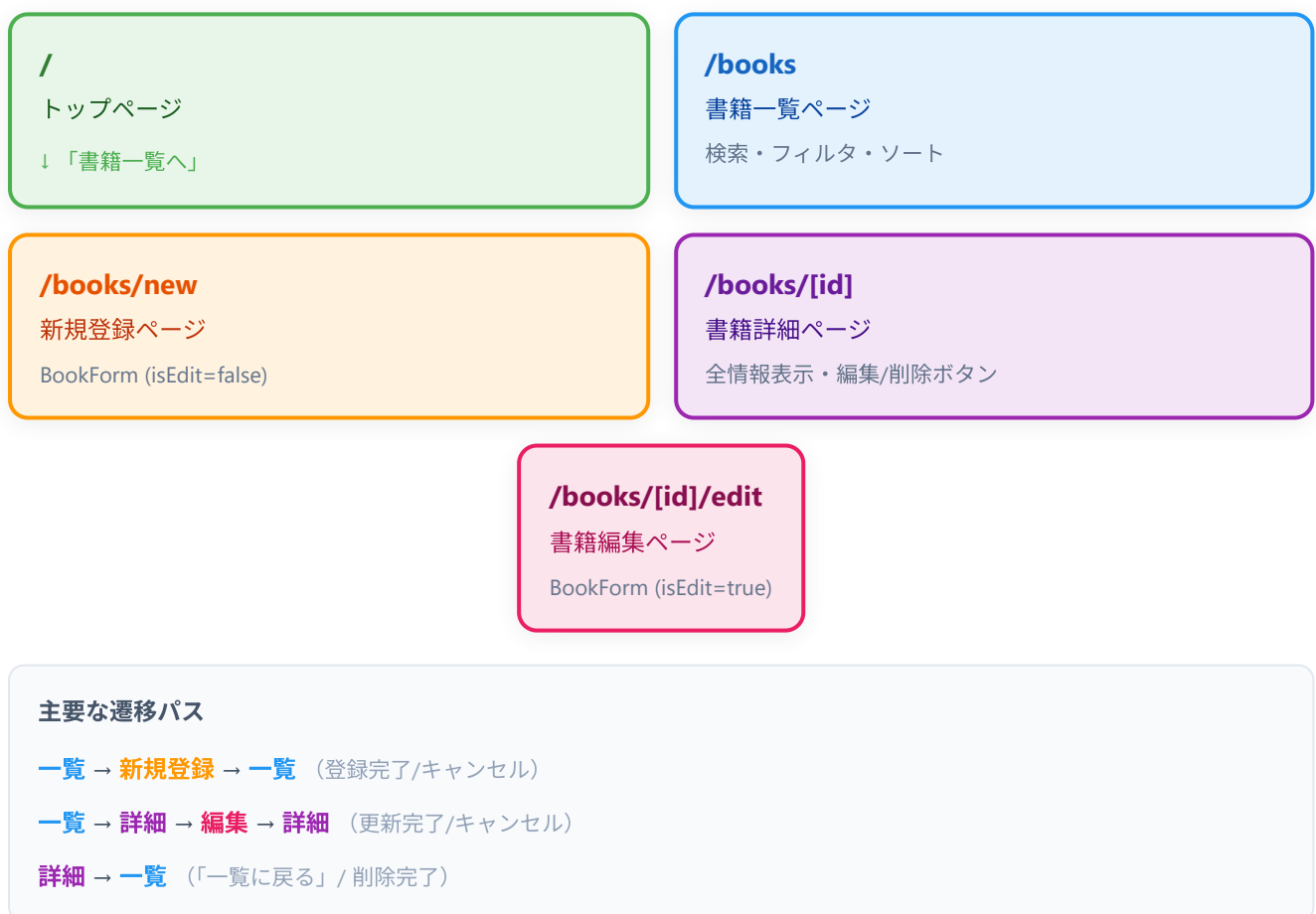
3. `null` 値の扱いに注意します。データベースから取得した値が `null` の場合、フォームの入力フィールドが uncontrolled になる可能性があります。

```
// null を空文字に変換
const initialData = {
  title: book.title ?? "", // null の場合は空文字
  author: book.author ?? "", // null の場合は空文字
  // ...
};
```

8. 全体のページ遷移図

最後に、書籍管理アプリ全体のページ遷移を確認します。以下の図は、ユーザーがアプリ内でどのようにページ間を移動するかを示しています。

書籍管理アプリ - ページ遷移図



この遷移図から、アプリの主要な導線を確認できます。

1. 一覧 → 新規登録 → 一覧: 新しい書籍を登録する流れ。登録完了後は一覧ページに戻り、登録した書籍がリストに追加されていることを確認できます。
2. 一覧 → 詳細 → 編集 → 詳細: 書籍の情報を編集する流れ。一覧から詳細を確認し、「編集する」ボタンで編集ページへ。編集完了後は詳細ページに戻り、更新内容を確認できます。
3. 一覧 → 詳細 → 削除 → 一覧: 書籍を削除する流れ。詳細ページで「削除する」ボタンを押すと確認ダイアログが表示され、確認後に削除が実行されて一覧ページに戻ります。

まとめ

この章では、以下の機能を実装しました。

機能	ファイル	説明
詳細表示	<code>app/books/[id]/page.tsx</code>	書籍の全情報を表示するページ
編集	<code>app/books/[id]/edit/page.tsx</code>	既存データをフォームに表示して更新
削除	<code>components/DeleteButton.tsx</code>	確認ダイアログ付きの削除ボタン
フォーム（共通）	<code>components/BookForm.tsx</code>	新規登録と編集の両方に対応
検索・フィルタ	<code>components/SearchBar.tsx</code>	テキスト検索とステータスフィルタ
ソート	<code>components/SortSelect.tsx</code>	複数のソート基準に対応

これで、書籍管理アプリの CRUD 機能がすべて揃いました。ユーザーは書籍の登録、表示、編集、削除を一通り行えるようになり、検索やフィルタ、ソートで効率的にデータを管理できます。

次の章では、認証機能を追加してユーザーごとにデータを分離する方法を学びます。